

### **Data Manipulation Language**

| Command       |                                                   |
|---------------|---------------------------------------------------|
| <b>INSERT</b> | Adds a new record to a table                      |
| <b>UPDATE</b> | Modifies the values of some records in the tables |
| <b>DELETE</b> | Remove records from the tables                    |
| <b>MERGE</b>  | Updates a table according to certain conditions   |
| <b>SELECT</b> | Find records in tables or views.                  |

*After updating the data, the command **SELECT \* FROM table\_name;** will be used to view the results!*

#### **ADDING RECORDS - INSERT**

Records can be added to tables in several ways:

- a) explicitly specifying the values, in the command syntax
- b) based on values from other tables
- c) with the help of substitution variables

**a) Adding records by explicitly specifying the values** can be done with the command:

**INSERT INTO table\_name VALUES** (list of values for each column in the order in which the columns in the table are declared);

Careful! Values must be specified for all columns of the table.

An alternative for this command is the following:

**INSERT INTO table\_name (column\_list) VALUES** (list of values for each column);

**Example 1.** Create the Workers table based on the Employees table without taking over the records (structure only) and add a new worker

```
CREATE TABLE Workers AS SELECT * FROM Employees WHERE 2=3;
INSERT INTO Workers (employee_id, last_name, salary) VALUES (207, 'Ionescu', 4000);
INSERT INTO Workers (employee_id, last_name, salary) VALUES (207, 'Popescu', 4200);
SELECT * FROM Workers;
```

**b) Adding records based on values from other tables:**

```
INSERT INTO table_name SELECT */column_list
FROM source_table
[WHERE condition];
```

**Example 2.** Add to the Workers table all employees in the Employees table who work in departments 20, 30 and 50. Complete the transaction (saving the changes).

```
INSERT INTO Workers SELECT * FROM Employees
WHERE department_id IN (20, 30, 50);
```

```
SELECT * FROM Workers;
```

```
COMMIT;
```

**c) Using substitution variables to add records based on user-entered values from the keyboard:**

```
INSERT INTO table_name (column_list) VALUES (&value_column1, &value_column2,...);  
SET DEFINE ON (for the environment to allow substitution variables)
```

**Example 3.** Add to the Workers table an employee whose data is entered by the user from the keyboard

```
INSERT INTO Workers (employee_id, last_name, hire_date, salary)  
VALUES ('& employee_id ', '&name', TO_DATE('&hire_date', 'mon dd yyyy'), '&salary');
```

*Attention! The TO\_DATE conversion function will be used for the hire\_date.*

*Ex: TO\_DATE('jan 20 2005', 'mon dd, yyyy')*

### **CHANGING DATA – UPDATE**

**The command syntax is:**

```
UPDATE table_name SET column_name = value  
[WHERE condition];
```

**Example 4.** Increase by 100 the salary of the workers who have a salary lower than 3000:

```
UPDATE Workers  
SET salary=salary+100  
WHERE salary<3000 AND commission_pct IS NULL;  
SELECT * FROM Workers;
```

**Example 5.** Update the salary of the workers whose manager has id = 122 with the salary of the worker with id = 125.

**Example 6.** Update the salary and commission of the ppl from the Workers table, with the salary and commission of the employee with employee\_id=167. Update only for those workers that work in department 50 and their salary is lower than the salary of employee\_id=174 from Employees

### **REMOVING DATA –DELETE**

**The command syntax is:**

```
DELETE FROM table_name  
[WHERE condition];
```

**Example 7.** Delete the workers from the Workers table who's manager\_id is equal to 122 or 123.

```
DELETE FROM Workers  
WHERE manager_id IN (122, 123);
```

**Example 8.** Remove all workers employed before 1999.

**Example 9.** Delete all employees from the Workers table. Cancel the transaction.

### **MERGE**

**The command syntax is:**

```
MERGE INTO table_name USING source_table  
ON (connection_condition)  
WHEN MATCHED THEN  
UPDATE SET column_name=value  
WHEN NOT MATCHED THEN  
INSERT VALUES (list_of_values);
```

**Example 10.** Update the Workers table so that all workers from this table have the same salaries as those in the Employees table, and for the Employees that are not workers, add the values of the columns (employee\_id, name, salary) from the Employees source table. Count the records in the two tables and explain the difference.

```
MERGE INTO workers USING employees  
ON (workers.employee_id = employees.employee_id)  
WHEN MATCHED THEN  
UPDATE SET workers.salary= employees.salary  
WHEN NOT MATCHED THEN  
INSERT (employee_id, last_name, salary, email, hire_date) VALUES (employees.  
employee_id, employees.  
last_name, employees.salary, employees.email, employees.hire_date);  
SELECT COUNT (*) FROM workers;  
SELECT COUNT (*) FROM employees;
```